



Politecnico  
di Torino

Rakuten  
Mobile

# Design and implementation of an eBPF-based runtime security monitoring Tool

---

End-to-End Pipeline and Hook Implementation

Simone Romano

# Project Reference and Architectural Constraints



## Tracee - Reference

- General-purpose, multi-kernel runtime security.
- Ring buffer as the primary transport channel.
- Reference for the probe registry, pipeline, and event model.

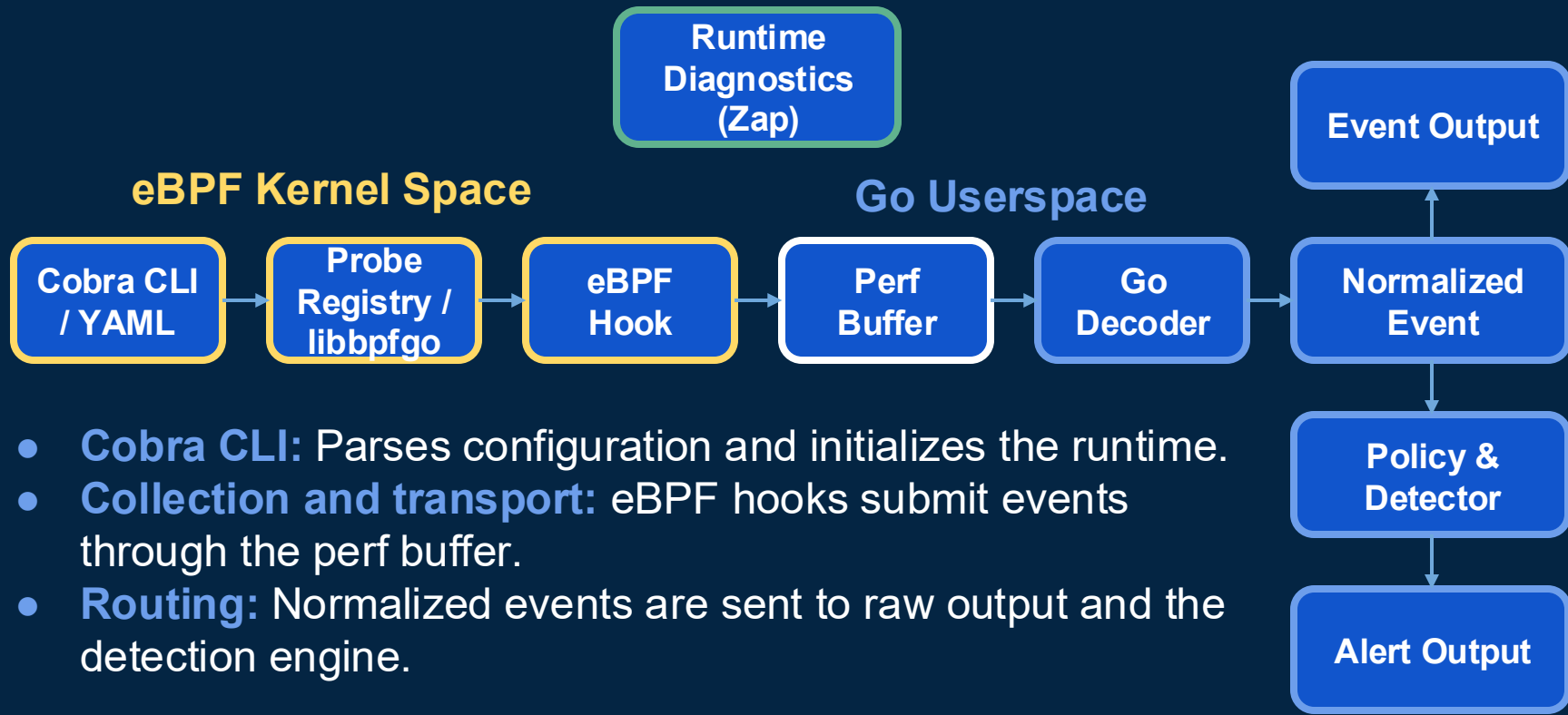


## Our Approach

- Targeted at Rocky Linux 8, kernel 4.18.
- Adapted to verifier constraints and provable memory offsets.
- Perf buffer as the main operational transport.
- Lightweight policy layer.

**Not a Tracee replica, but a more focused and controllable runtime.**

# End-to-End Architecture and Pipeline



# Binary Contract and Event Normalization



From binary records

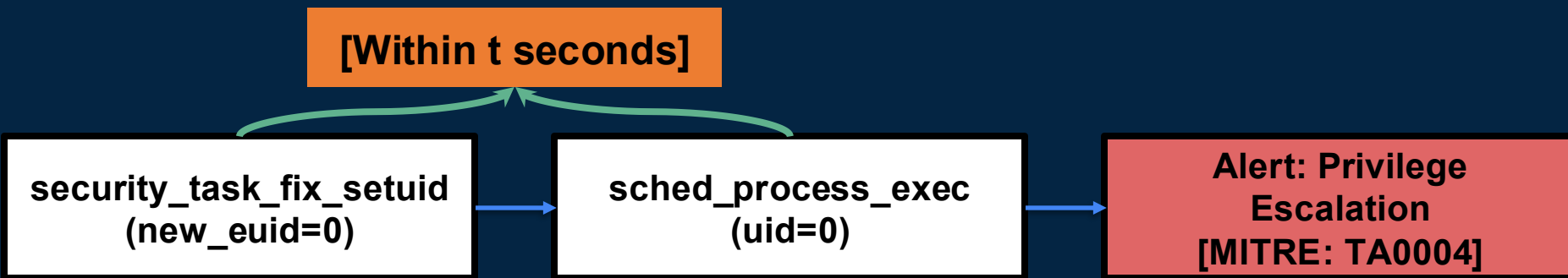
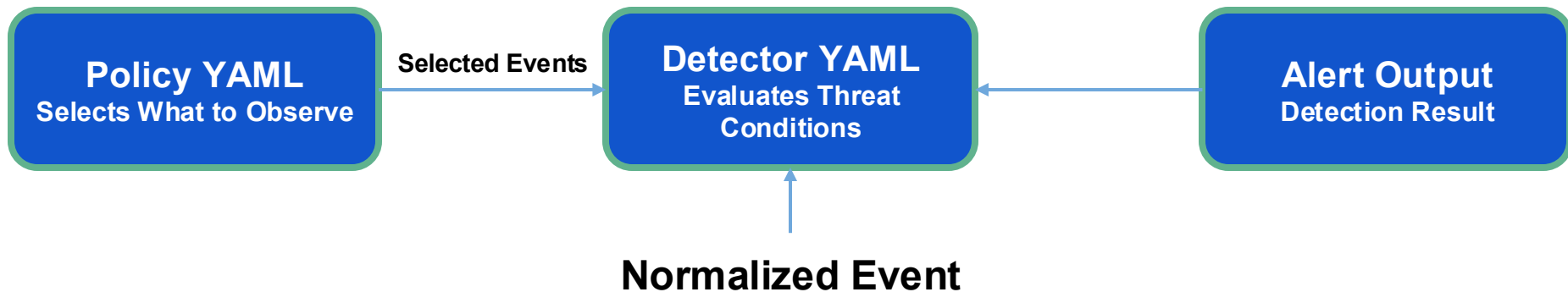


to typed, indexed arguments



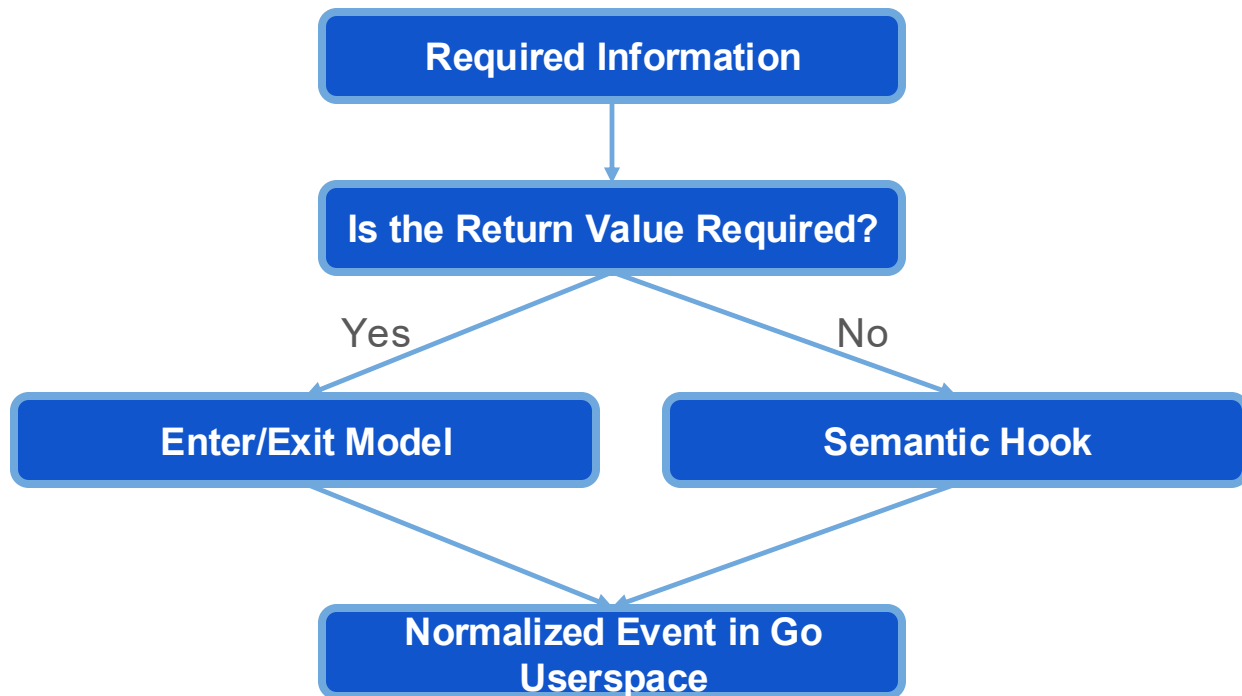
- **Binary contract:** Shared context, indexed arguments, and synchronized C/Go registries.
- **Unique mapping:** eBPF event ID, event name, and decoding schema.
- **Enrichment:** Userspace normalization of capabilities, flags, bitmasks, and errno values.

# Policy and Detection Layer (YAML)

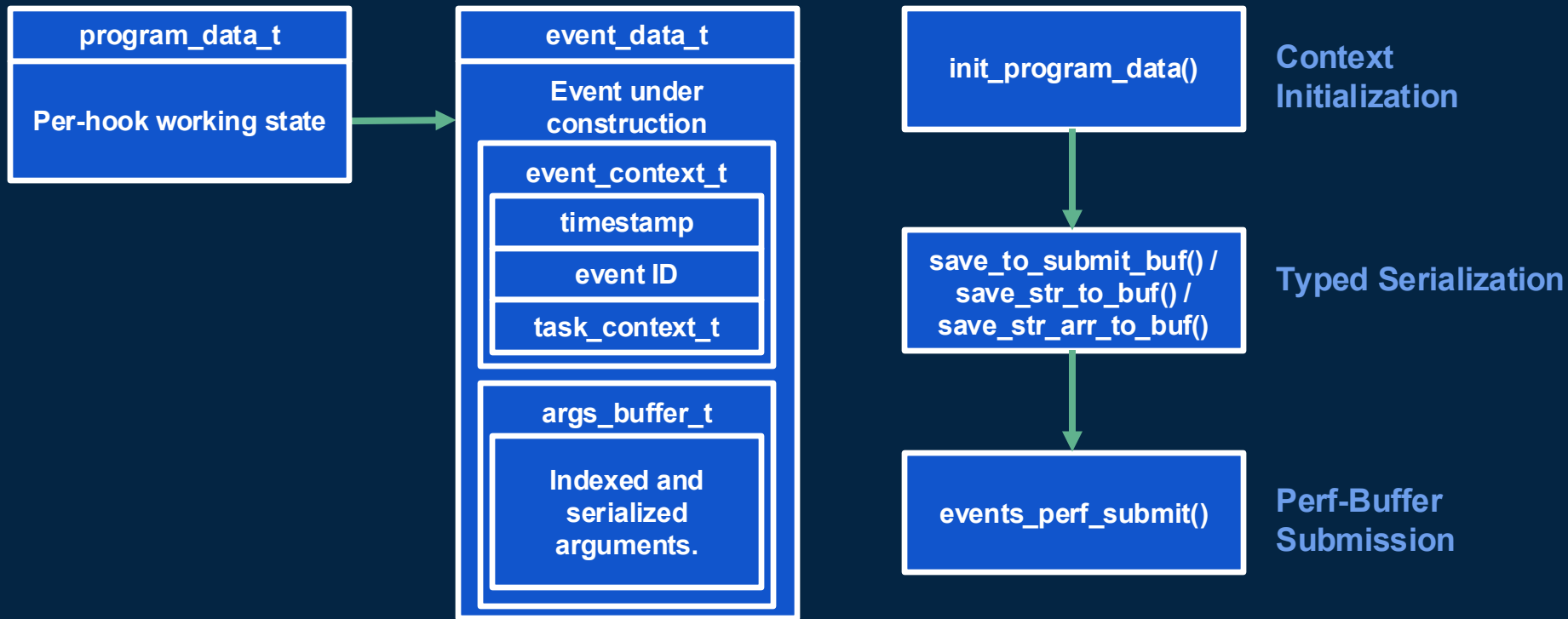


# Kernel Hook Implementation Strategy

- **Semantic hooks:**  
Primarily based on kprobes and security hooks exposing resolved kernel objects.
- **Enter/exit model:** Entry stores arguments; exit adds the final return value.
- **Kernel adaptation:**  
Designed around Rocky Linux 4.18 verifier constraints.



# Common eBPF Implementation Pattern



When the return value is required, entry stores the arguments and exit completes the event with the syscall outcome.

# Process Lifecycle

**fork / vfork / clone**

**Syscall Request:**  
Capture the initial  
process-creation request.

**task\_rename**

**Task transition:**  
Track changes to the  
process comm.

**sched\_process\_exit**

**Termination:**  
Record process exit and  
exit code.

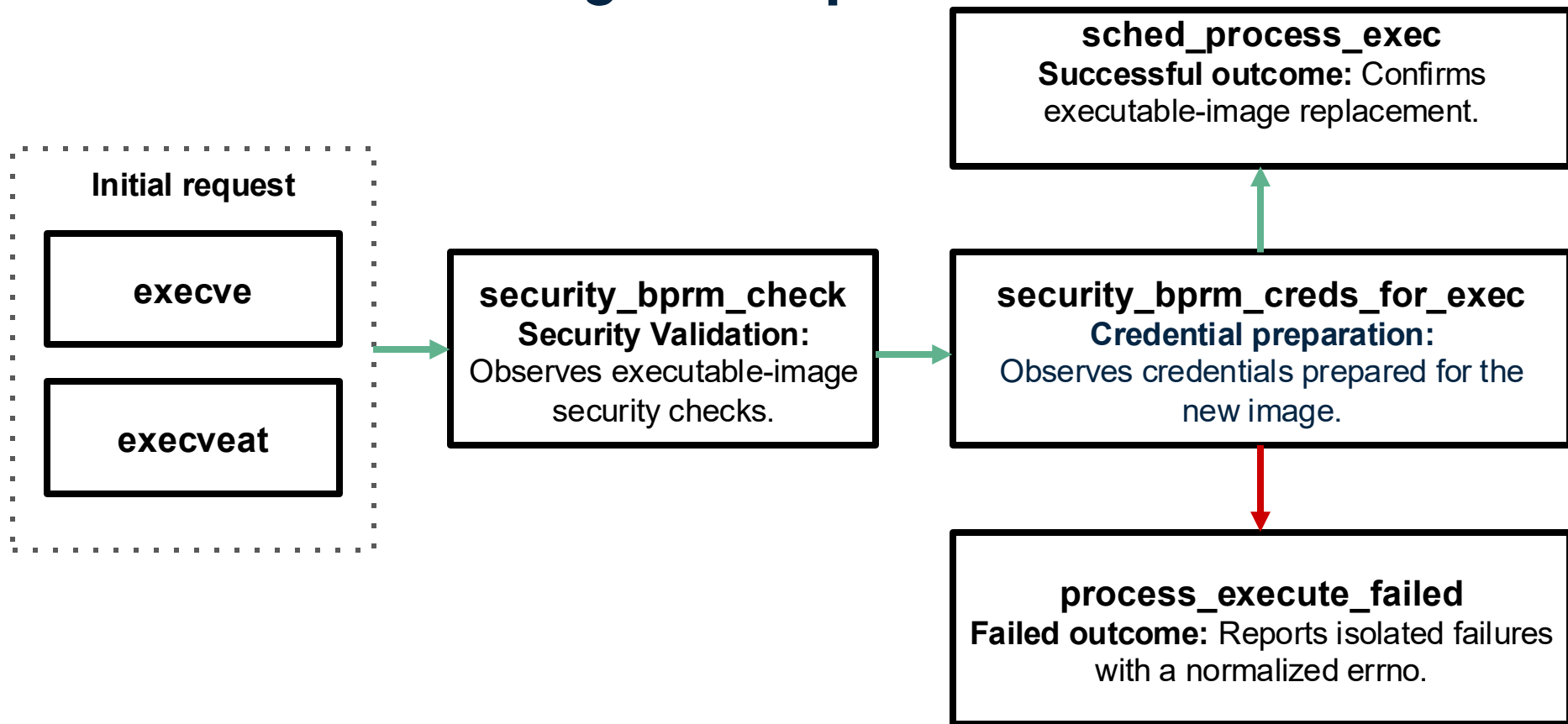
**sched\_process\_fork**

**Parent-child  
relationship:**  
Observe the task  
relationship established  
by the kernel.

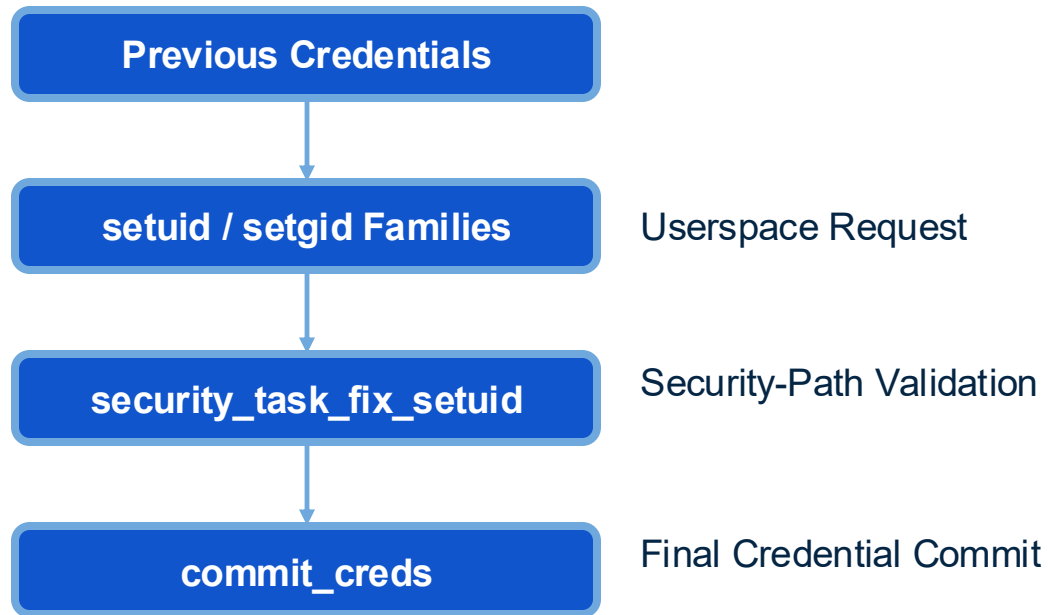
**sched\_process\_exec**

**Executable image:**  
Confirm successful image  
replacement.

# Execution Monitoring: Attempts and Outcomes



# Identity, Credentials, and Capabilities



## Capability and process control

- **cap\_capable:** Observes checks for specific capabilities.
- **ptrace** and **security\_task\_prctl:** Observe direct process interaction and state manipulation.

# File, Permission, and Filesystem Security

## File Access

- **open**: reports the syscall outcome.
- **security\_file\_open** / **security\_file\_permission**: expose stable metadata (path, inode, device, and ctime)

## Permissions

- The **chmod** and **chown** families monitor permission and ownership changes, including return values.

## Inode

- **unlink**, **rename**, **symlink** and **mknod**: track structural filesystem changes.

## Mounting

- **security\_sb\_mount** / **security\_sb\_unmount**: observe mount points and associated flags.

# Memory and Namespaces

## Memory Management

*mmap*

*mprotect*

*pkey\_mprotect*

*memfd\_create*

*process\_vm\_readv* / *process\_vm\_writev*

Visibility into executable-memory transitions, fileless execution, unusual mappings, and cross-process memory access.

---

## Namespace Transitions

*setns*

*unshare*

*switch\_task\_ns*

Observes isolation requests and verifies the namespace changes effectively applied by the kernel.

# Kernel Hardening and Modules

**Module lifecycle:** *module\_load,*  
*module\_free,*  
*do\_init\_module.*

---

**eBPF surface:** *security\_bpf,*  
*security\_bpf\_map,*  
*security\_bpf\_prog.*

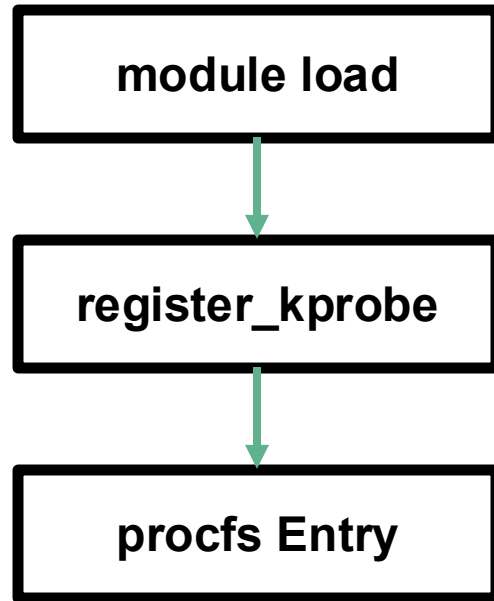
---

**Dynamic instrumentation:** *register\_kprobe,*  
*kallsyms\_lookup\_name.*

---

**Kernel interfaces:** *proc\_create,*  
*call\_usermodehelper,*  
*security\_kernel\_read\_file.*

Example of a scenario  
observable through distinct  
events



# Experimental Validation Setup

## Monitor Pod

eBPF Agent

Filtering on UID  
10001

## Trigger Pod

Exploit Client  
(Attacker)

Attack sequence:

Exploit -> ROP chain

-> /bin/sh -> /bin/cat ->

File Exfiltration

## Target Pod

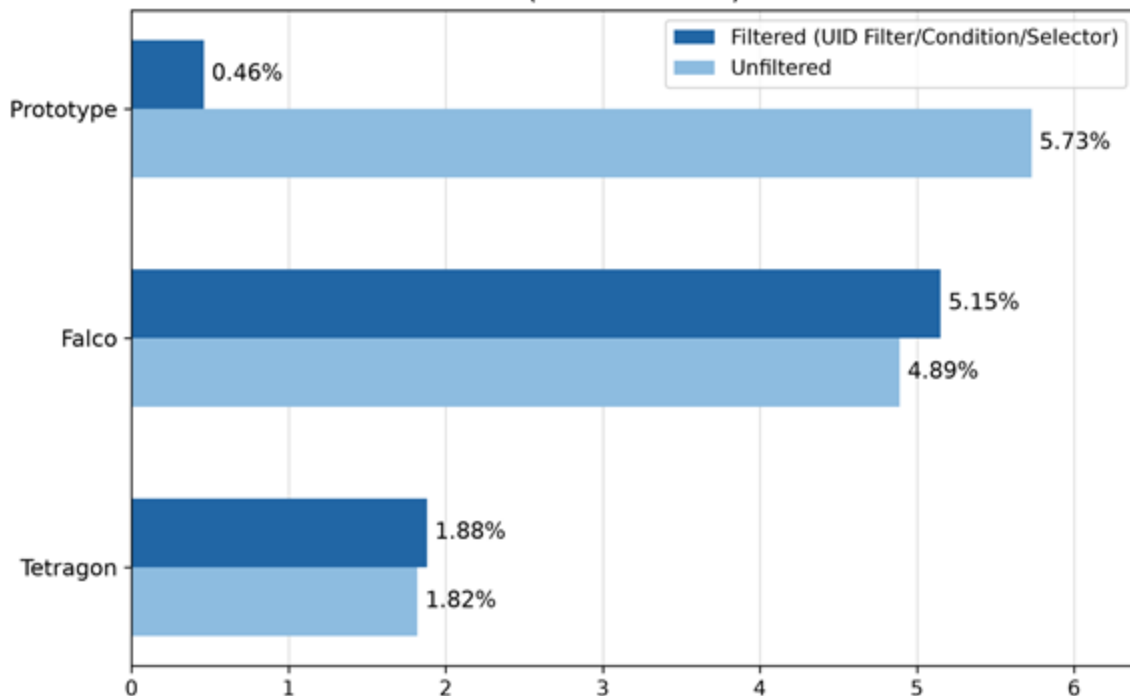
Vulnerable CTF  
Service (UID  
10001)

Contains Flag  
Secret

Shared Linux Kernel (Rocky Linux 8.10)

# Evaluation Results: CPU Consumption

Mean Monitor-Container CPU Consumption  
(% of one core)



- The prototype reduced mean CPU by about 92% through filtering before transport.
- All retained runs recovered the flags and produced the required evidence.
- Falco, Tetragon, and the prototype apply **UID** selection at different stages

# Conclusions

## Key Findings

- **Kernel observations can be transformed into consistent security evidence.**
- **Configurable monitoring supports different investigation task, useful for the security analyst.**
- **Early filtering can reduce performance cost**
- **Broader coverage and higher event rates remain future work.**

**Any question?**